

15 · Riesgos y deuda técnica

Documentacion del sistema

Version analizada: **0.3.0**

Commit analizado: **ff246ab**

Fecha de generacion: **2026-08-27**

Generado a partir de 15-risks-and-technical-debt.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Resumen
2. Severidad alta
3. Severidad media
4. Severidad baja
5. Decisiones que requieren validación humana
6. Lo que está bien y hay que proteger
7. Plan sugerido, por esfuerzo e impacto

Documento informativo. No se corrigió ninguno de estos hallazgos. Cada uno lleva severidad, impacto, probabilidad, evidencia verificable, ubicación, recomendación y prioridad. La sección final recoge lo que está bien y conviene proteger, porque un informe que solo enumera problemas da una imagen falsa.

1. Resumen

Severidad	Cantidad	Hallazgos
[alto] Alta	6	R-01 ... R-06
[medio] Media	12	R-07 ... R-18
[bajo] Baja	9	R-19 ... R-27
Total	27	

Categoría	Cantidad
Seguridad	7
Corrección funcional	6
Empaquetado y despliegue	3
Consistencia y duplicación	5
Operación y escalabilidad	4
Documentación	2

Todos los hallazgos se verificaron leyendo el código o ejecutando un comando. Los marcados **REPRODUCIDO** se ejecutaron durante el análisis; los marcados **INFERENCIA** dependen de una condición que no se pudo montar.

2. Severidad alta

R-01 · Los flows 21–27 probablemente no funcionan en el binario empaquetado

Campo	Valor
Severidad	[alto] Alta
Categoría	Empaquetado
Impacto	Siete de los 27 flows —la familia web completa, la novedad de la v0.3.0— fallarían en el producto distribuido
Probabilidad	Alta INFERENCIA
Ubicación	<code>installer/automa.spec</code> , lista <code>hiddenimports</code>

Evidencia. `grep -c "browser_extract" installer/automa.spec` devuelve 0. La lista enumera `actions.filesystem`, `screen`, `vision`, `system`, `rules`, `ui`, `http_actions`, `notify`, `browser_capture` y `browser_form` — diez módulos. Falta `actions.browser_extract`, añadido en la v0.3.0.

`LazyActionRegistry.get` resuelve con `import_module(module_name)`, un import **dinámico** que el analizador estático de PyInstaller no rastrea.

Recomendación. Añadir `"actions.browser_extract"` a `hiddenimports`. Mejor aún: derivar la lista de `_BUILT_IN_ACTIONS` para que nunca vuelva a desincronizarse.

REQUIERE VALIDACIÓN: confirmar compilando y ejecutando el flow 21 desde el `.exe`.

Prioridad: 1.

R-02 · `GET /api/runs` expone todo el histórico sin autenticación

Campo	Valor
Severidad	[alto] Alta
Categoría	Seguridad · privacidad
Impacto	Portapapeles capturado, texto OCR de ventanas abiertas, contenido web e inventarios del equipo, legibles sin token
Probabilidad	Baja en loopback con un solo usuario; alta si se expone el panel
Ubicación	<code>app/server.py::do_GET</code>

Evidencia. `do_GET` no llama a `_authorize_mutation` en ningún caso. Verificado leyendo el método completo. Con `AUTOMA_PANEL_TOKEN` definido, siguen abiertos `/api/runs`, `/api/flows`, `/api/metrics`, `/metrics`, `/file` y todas las vistas HTML.

`GET /api/runs` devuelve la columna `context_json` completa de cada corrida. Y `/file` sirve los `.png` de las capturas: la `allowlist` de extensiones bloquea `.html`, `.js`, `.svg` y `.css`, **pero no** `.png`.

Recomendación. Cuando `AUTOMA_PANEL_TOKEN` esté definido, exigirlo también en los GET que devuelven datos (`/api/runs`, `/api/flows`, `/api/metrics`, `/file`). Es una llamada adicional en `do_GET`.

Prioridad: 2.

R-03 · 14 de 27 flows corren sin ninguna política de sandbox

Campo	Valor
Severidad	[alto] Alta
Categoría	Seguridad
Impacto	La mitad del catálogo puede ejecutar cualquier acción y escribir en cualquier ruta
Probabilidad	Cierta: es el estado actual
Ubicación	<code>flows/01-07/manifest.json</code> , <code>flows/21-27/manifest.json</code>

Evidencia. Conteo leyendo los 27 manifests:

Control	Lo declaran	No lo declaran
<code>allowed_actions</code>	13 (08–20)	14

<code>allowed_paths</code>	7	20
<code>max_runtime_seconds</code>	13	14
<code>required_secrets</code>	0	27

Entre los 14 sin política están `07_browser_form_filler` (lanza un navegador visible y escribe archivos) y los siete de la familia web.

INFERENCIA: el bloque 08–20 se añadió en la v0.2.0 con la política ya en mente y la familia web de la v0.3.0 no la incorporó. Nada en la CI lo exige, así que el hueco puede crecer con cada caso nuevo.

Recomendación. Añadir a `scripts/validate_project.py` una regla que exija `allowed_actions` en todo flow. Retroadaptar los 14 existentes.

Prioridad: 3.

R-04 · `system.run_powershell` permite ampliar su allowlist desde el manifest

Campo	Valor
Severidad	[alto] Alta
Categoría	Seguridad
Impacto	Un manifest puede habilitar verbos destructivos de PowerShell
Probabilidad	Baja: requiere escribir un manifest, es decir, acceso de escritura
Ubicación	<code>actions/system.py::run_powershell</code> , parámetro <code>allowlist</code>

Evidencia.

```
verbs = tuple(allowlist) if allowlist else _PS_DEFAULT_ALLOWLIST
```

`allowlist` es un parámetro de la acción y, por tanto, sobrescribible desde `params`. Un flow con `"allowlist": ["Remove-Item"]` obtiene exactamente eso. Los tokens prohibidos (`;`, `|`, `&...`) no impiden un comando destructivo de una sola palabra con argumentos.

Contexto que matiza la gravedad. No es explotable desde fuera: quien puede escribir un manifest ya tiene acceso al sistema de archivos. Es precisamente el modelo de amenaza que `security.yml` describe, y por eso el hardening del CI es la defensa principal. Pero significa que la seguridad de esta acción depende de la revisión humana de los manifests, no de un control del motor.

Recomendación. Eliminar el parámetro `allowlist` de la firma pública, o exigir que el conjunto ampliado sea subconjunto de una lista maestra.

Prioridad: 4.

R-05 · `allowed_actions: []` produce política permisiva, no restrictiva

Campo	Valor
Severidad	[alto] Alta
Categoría	Seguridad · corrección
Impacto	Un autor que escriba una lista vacía creyendo bloquearlo todo obtiene lo contrario

Probabilidad	Media
Ubicación	engine/loader.py::load_manifest

Evidencia.

```
allowed_actions=list(raw['allowed_actions']) if raw.get('allowed_actions') else None
allowed_paths=list(raw['allowed_paths']) if raw.get('allowed_paths') else None
```

Una lista vacía es *falsy* en Python, así que `[]` → `None`, y `None` significa «sin restricción» en `SandboxPolicy`. Es un fallo silencioso con impacto de seguridad, sin ninguna prueba que lo cubra.

Recomendación. Cambiar la condición a `is not None` en ambos campos, y añadir una prueba de regresión.

Prioridad: 5.

R-06 · El CLI falla al imprimir en una consola Windows por defecto **REPRODUCIDO**

Campo	Valor
Severidad	[alto] Alta
Categoría	Corrección funcional
Impacto	automa list y automa run terminan con excepción y código de error en Windows
Probabilidad	Cierta: reproducido en este análisis
Ubicación	engine/runner.py::main, líneas 39 y 41

Evidencia reproducida.

```
$ python -m engine.runner list
UnicodeEncodeError: 'charmap' codec can't encode character '→' in position 6042:
character maps to <undefined>
```

Ocorre con `list` y con `run`. La causa es `print(json.dumps(..., ensure_ascii=False, indent=2))` sobre una consola `cp1252`; varias descripciones de flow contienen →.

Matiz importante: con `run`, el flow se ejecuta completo y correctamente —la corrida queda persistida y el reporte escrito— y el error salta después, solo al volcar el JSON.

Verificado que la variable de entorno lo resuelve: con `PYTHONIOENCODING=utf-8`, el comando devuelve el JSON completo.

Recomendación. Escribir a `sys.stdout.buffer` con codificación UTF-8 explícita, o reconfigurar `sys.stdout` al inicio de `main()`. Añadir una prueba que capture la salida con codificación forzada — `engine/runner.py` está al **0 % de cobertura**, y una sola prueba habría detectado esto.

Prioridad: 6.

3. Severidad media

R-07 · El lock de ejecución solo cubre una de las cuatro vías de disparo

Campo	Valor
Severidad	[medio] Media

Impacto	El mismo flow puede correr dos veces en paralelo desde el panel, con escritura concurrente de los archivos de tracking (flows 07, 23, 26)
Probabilidad	Media
Ubicación	<code>engine/database.py::acquire_run_lock</code> , <code>app/server.py::do_POST</code>

Evidencia. Búsqueda en todo el repositorio: `acquire_run_lock` se invoca **solo** desde `engine/scheduler.py::_run_job`. Ni `POST /api/run/<folder>`, ni `POST /run`, ni `POST /api/hook/<folder>`, ni el CLI lo usan.

Recomendación. Aplicar el lock también en las rutas del panel y del webhook, con una respuesta **409** cuando esté tomado.

Prioridad: 7.

R-08 · No hay retención: los almacenes crecen sin límite

Campo	Valor
Severidad	[medio] Media
Impacto	Degradación del panel y de las métricas; consumo de disco
Probabilidad	Cierta a medio plazo
Ubicación	<code>engine/database.py</code> , <code>engine/logger.py</code> , <code>engine/state_store.py</code>

Evidencia. Búsqueda de `VACUUM`, `DELETE FROM runs`, `retention` y `backup` en todo el código: **no existe ninguna rutina**. Y no hay ningún `CREATE INDEX`: solo los índices implícitos de las claves primarias. `list_runs(flow_id=...)`, `get_steps(run_id)` y `get_events(run_id)` hacen escaneo completo.

INFERENCIA: un flow programado cada 15 minutos genera 96 corridas diarias, ~35 000 al año, con ~70 000 archivos sueltos entre `logs/` y `state/`.

Recomendación. Índices sobre `runs.flow_id`, `runs.created_at`, `steps.run_id` y `events.run_id`, más una rutina de purga configurable.

Prioridad: 8.

R-09 · `max_runtime_seconds` no interrumpe una acción en curso

Campo	Valor
Severidad	[medio] Media
Impacto	Un flow con límite de 30 s puede correr indefinidamente si un solo paso se cuelga
Probabilidad	Media
Ubicación	<code>engine/orchestrator.py::run</code>

Evidencia. El límite se comprueba **antes** de lanzar cada paso, comparando el tiempo transcurrido. Una acción que se bloquea dentro no se interrumpe. Afecta a los 13 flows que declaran el campo, incluidos `18_powershell_audit` (40 s) y `12_desktop_ocr_inventory` (30 s).

Recomendación. Ejecutar la acción en un `concurrent.futures.ThreadPoolExecutor` con `timeout`, o documentar explícitamente que es un control de arranque de paso.

Prioridad: 9.

R-10 · El scheduler se traga los errores sin ninguna señal

Campo	Valor
Severidad	[medio] Media
Impacto	Una tarea programada que falla siempre reprograma con normalidad y nadie se entera
Probabilidad	Media
Ubicación	engine/scheduler.py::_run_job

Evidencia.

```
try:
    Orchestrator(Path(flow['flow_path'])).run()
except Exception:
    pass # "No interrumpe el loop. El error queda en la corrida."
mark_schedule_run(folder, interval_seconds, cron_expression)
```

`mark_schedule_run` se llama **después** del `except`, es decir, también cuando el flow falla. No hay traza en consola —`log_message` está silenciado— ni alerta.

Recomendación. Registrar el fallo en el log del proceso y, opcionalmente, contar fallos consecutivos para deshabilitar la tarea tras N.

Prioridad: 10.

R-11 · engine/catalog.py duplica root_dir() e ignora el modo empaquetado

Campo	Valor
Severidad	[medio] Media
Impacto	En el binario, <code>catalog.flows_dir()</code> no apuntaría a los flows extraídos por PyInstaller
Probabilidad	Media INFERENCIA
Ubicación	engine/catalog.py, líneas 10–16

Evidencia.

```
# engine/catalog.py – definición local
def root_dir() -> Path:
    return Path(__file__).resolve().parent.parent
```

`engine/paths.py::root_dir` sí contempla `$AUTOMA_ROOT` y `sys._MEIPASS`. La de `catalog.py` no. En desarrollo coinciden; congelado, no.

INFERENCIA matizada: `installer/automa_entry.py` define `AUTOMA_ROOT` y el bundle extrae los flows bajo `_MEIPASS`, que es también el directorio de `engine/`, así que podrían coincidir por accidente. **REQUIERE VALIDACIÓN** compilando.

Recomendación. Importar `root_dir` de `engine.paths` y borrar la definición local.

Prioridad: 11.

R-12 · configs/ se lee relativo al cwd pero se escribe en data_dir()

Campo	Valor
-------	-------

Severidad	[medio] Media
Impacto	La configuración guardada desde el panel puede no leerse al ejecutar el flow
Probabilidad	Media
Ubicación	<code>engine/loader.py::load_context</code> frente a <code>engine/database.py::set_flow_config</code>

Evidencia.

```
# loader.py – relativo al directorio de trabajo
Path('configs') / f'{flow_dir.name}.json'
# database.py – absoluto sobre la raíz escribible
config_path = data_dir() / 'configs' / f'{folder}.json'
```

En desarrollo coinciden si se ejecuta desde la raíz. `installer/automa_entry.py` hace `os.chdir(data_dir())` para que coincidan en el bundle. Pero cualquier ejecución del CLI desde otra carpeta rompe la resolución.

Recomendación. Usar `data_dir() / 'configs'` también en `load_context`.

Prioridad: 12.

R-13 · `required_secrets` solo mira el entorno, no la bóveda

Campo	Valor
Severidad	[medio] Media
Impacto	Un secreto guardado con <code>set_secret</code> no satisface <code>required_secrets</code> ; el flow no arranca
Probabilidad	Media si alguien empieza a usar el campo
Ubicación	<code>engine/sandbox.py::check_required_secrets</code>

Evidencia. Usa `os.environ.get(name)`, no `engine.secrets.get_secret`. Los tokens del panel sí se resuelven desde el archivo; `required_secrets` no. Inconsistencia entre dos mecanismos presentados como equivalentes.

Atenuante: ningún flow del catálogo declara `required_secrets`, así que hoy no afecta a nadie.

Recomendación. Usar `get_secret` en `check_required_secrets`.

Prioridad: 13.

R-14 · Dos motores de condiciones con capacidades distintas

Campo	Valor
Severidad	[medio] Media
Categoría	Duplicación
Impacto	Un operador que funciona en un <code>when</code> falla dentro de <code>rules.evaluate</code>
Probabilidad	Media
Ubicación	<code>engine/conditions.py::matches</code> frente a <code>actions/rules.py::_matches</code>

Evidencia.

Motor	Operadores
<code>engine/conditions.py</code> (when de pasos y transiciones)	13: <code>eq, ne, gt, gte, lt, lte, contains, in, exists, not_exists, truthy, falsy, regex</code>
<code>actions/rules.py</code> (<code>rules.evaluate</code>)	6: <code>eq, ne, gt, lt, contains, in</code>

Además, `contains` normaliza a minúsculas en el primero (`str(expected).lower() in str(actual).lower()`) y **no** en el segundo (`str(expected) in str(value)`). El mismo nombre de operador se comporta distinto según dónde se use.

Recomendación. Hacer que `actions/rules.py` delegue en `engine.conditions.matches`.

Prioridad: 14.

R-15 · `_smart_summary` solo cubre 6 de los 27 flows, y el README afirma otra cosa

Campo	Valor
Severidad	[medio] Media
Categoría	Documentación · experiencia de uso
Impacto	El detalle de 21 flows muestra solo JSON crudo; el README promete lo contrario
Probabilidad	Cierta
Ubicación	<code>app/server.py::_smart_summary</code> , <code>README.md</code>

Evidencia. La cadena de `elif` tiene rama para `screen_capture_analyze`, `screen_capture_browser`, `folder_inventory`, `document_drop_pipeline`, `system_healthcheck` y `process_watchdog`. Para el resto devuelve cadena vacía y `render_run_detail` omite el bloque entero.

El `README.md` afirma, en su demo de cinco minutos: «click el último run del flow 07 → verás los 10 datos enviados como **lista legible** (no JSON crudo)». **No hay rama para `browser_form_filler`.**

Recomendación. Añadir ramas para los flows más usados, o corregir el README.

Prioridad: 15.

R-16 · La cobertura no es reproducible entre ejecuciones **REPRODUCIDO**

Campo	Valor
Severidad	[medio] Media
Impacto	La CI podría fallar sin ningún cambio de código
Probabilidad	Baja pero real: el margen sobre el gate es < 5 puntos
Ubicación	<code>app/server.py</code> (líneas de módulo), <code>pyproject.toml</code>

Evidencia reproducida. Dos ejecuciones consecutivas de `python -m pytest` en el mismo commit dieron **58,92 %** y **60,0 %**. `engine/scheduler.py` marcó **51 %** en una y **76 %** en la otra.

INFERENCIA de la causa: importar `app.server` arranca un hilo de scheduler (`SCHEDULER.start_in_background()`) a nivel de módulo). Cuántas líneas de `scheduler.py` se ejecutan depende del temporizado del hilo. El gate está en 54 %.

Recomendación. No arrancar el scheduler al importar: moverlo a `run_server()`, o condicionarlo a una variable de entorno que los tests no definan.

Prioridad: 16.

R-17 · Importar `app.server` tiene cuatro efectos secundarios

Campo	Valor
Severidad	[medio] Media
Impacto	Cualquier <code>import app.server</code> levanta un hilo, crea la base y sincroniza el catálogo
Probabilidad	Cierta
Ubicación	<code>app/server.py</code> , líneas 100–104

Evidencia.

```
SCHEDULER = SchedulerService(loop_sleep_seconds=2.0)
SCHEDULER.start_in_background()
init_db()
sync_flows(list_flows())
```

Se ejecutan al **importar** el módulo, no al llamar a `run_server()`. Afecta a `tests/test_panel_endpoints.py` y es la causa de R-16.

Recomendación. Mover los cuatro a `run_server()`.

Prioridad: 17.

R-18 · No hay lockfile de dependencias versionado

Campo	Valor
Severidad	[medio] Media
Categoría	Seguridad · reproducibilidad
Impacto	Dos instalaciones pueden resolver versiones distintas; un escáner de CVE no puede pronunciarse con precisión
Probabilidad	Cierta
Ubicación	Raíz del repositorio

Evidencia. `git ls-files` no incluye `uv.lock`, `requirements.lock`, `poetry.lock` ni `Pipfile.lock`. El comentario de `requirements.txt` lo reconoce: «Para reproducibilidad estricta usar `uv export` (lockfile en CI) — este archivo solo fija un piso seguro».

Atenuante fuerte: `pyproject.toml` fija un piso explícito por paquete con motivo documentado («primera versión sin CVE conocida al corte de la auditoría 2026-06-01») y una cota superior de *major*. Es mejor que un rango abierto.

Recomendación. Generar y versionar `uv.lock`.

Prioridad: 18.

4. Severidad baja

R-19 · requirements.txt está incompleto

Severidad [bajo] · Ubicación requirements.txt frente a pyproject.toml

Declara Pillow, psutil, requests, mss, pyautogui y pytessearact. Le faltan pyperclip, PyGetWindow y pywebview. make install usa ese archivo y deja el entorno sin la ventana nativa ni los flows 15 y 16. Además, el archivo no termina en salto de línea.

Recomendación. Sincronizarlo o eliminarlo y apuntar solo a pyproject.toml. Prioridad: 19.

R-20 · pyproject.toml declara 31 entry points frente a 36 acciones registradas

Severidad [bajo] · Ubicación pyproject.toml, [project.entry-points."automa.actions"]

Faltan browser.capture_page, browser.crawl_site, browser.extract_content, browser.fill_form y http.check_urls. Verificado programáticamente. No rompe el runtime —LazyActionRegistry consulta primero su diccionario interno— pero un paquete externo que inspeccione el grupo verá un catálogo incompleto.

Recomendación. Sincronizar, o generar la sección desde BUILT_IN_ACTIONS. Prioridad: 20.

R-21 · Código muerto en decision/

Severidad [bajo] · Ubicación decision/rules.py, decision/optional_ai.py

prioritize_steps y suggest_step_order devuelven la lista sin tocarla y nadie los importa. Verificado con búsqueda en todo el repositorio. El paquete sí aparece en [tool.hatch.build.targets.wheel].packages y en --cov=decision, donde figura al 0 %, arrastrando la cobertura global hacia abajo.

Matiz: la docstring de optional_ai.py documenta una decisión arquitectónica que sigue siendo válida —«La IA solo debe sugerir orden o prioridad, nunca reemplazar la ejecución del motor»— aunque el código no exista.

Recomendación. Eliminar el paquete y conservar la decisión en un ADR, o marcarlo explícitamente como punto de extensión y sacarlo de --cov. Prioridad: 21.

R-22 · Tres divergencias silenciosas del cron con crontab(5)

Severidad [bajo] · Ubicación engine/cron.py

Divergencia	Aquí	crontab(5)
Día de la semana	0 = lunes	0 = domingo
Combinación día-mes / día-semana	AND	OR cuando ninguno es *
Zona horaria	Siempre UTC	Hora local del sistema

La primera está documentada en un comentario del código; las otras dos no. Ninguna aparece en docs/OPERACION.md ni en la ayuda del panel. Una expresión copiada de internet se ejecutará el día o la hora equivocados sin ningún error.

Recomendación. Documentarlas en la ayuda del panel y en docs/OPERACION.md. Considerar alinear el día de la semana con el estándar. Prioridad: 22.

R-23 · Diez acciones registradas sin ningún flow que las use

Severidad [bajo] · Ubicación actions/

De las 36 registradas, 26 aparecen en algún manifest. Las otras diez son filesystem.ensure_directory, read_text_file, move_file, http.fetch_url, ui.open_file_in_browser, ui.click, ui.click_bbox, vision.select_image, find_text_in_image e inspect_screen_target.

No son código muerto —forman parte del contrato público del registro— pero son superficie sin cobertura de caso. `vision.inspect_screen_target` es la más relevante: es el **único camino** hacia `VisionModelAnalyzer`, y al no usarla ningún flow, el catálogo publicado no puede llamar a un proveedor de IA.

Recomendación. Escribir un caso que las ejercite, o marcarlas como experimentales. **Prioridad: 23.**

R-24 · Cotas no declaradas en `summarize_text_folder` y `apply_tracking`

Severidad [bajo] · **Ubicación** `actions/filesystem.py`, `actions/browser_extract.py`

El sistema declara nueve de sus trece cotas (`truncated`, `links_truncated`, `text_truncated`, `stdout_truncated`...). Dos no lo hacen:

1. `summarize_text_folder` corta en `max_files` (10 por defecto) y solo acepta cinco extensiones. El resultado no dice cuántos archivos quedaron fuera.
2. `apply_tracking` trata un JSON de estado corrupto como inexistente y **reinicia la línea base en silencio**, reportando `first_run: true` sin explicar por qué.

Recomendación. Añadir `skipped_count/truncated` al primero y un campo `state_reset_reason` al segundo. **Prioridad: 24.**

R-25 · `app/server.py` concentra 1 753 líneas con cuatro responsabilidades

Severidad [bajo] · **Ubicación** `app/server.py`

El 29 % del Python de producción, con enrutado HTTP, ~430 líneas de CSS, ~660 de renderizado HTML por concatenación de f-strings, y la lógica de autorización. Cobertura: **38 %**, la más baja de los módulos grandes.

El riesgo concreto: el escapado HTML se aplica con `html.escape` en las rutas revisadas, pero **una ruta sin escapar sería difícil de detectar por lectura** en 660 líneas de f-strings.

Recomendación. Separar al menos la hoja de estilo y las plantillas en archivos estáticos servidos desde disco. **Prioridad: 25.**

R-26 · `scripts/smoke_test.py` modifica un archivo versionado **REPRODUCIDO**

Severidad [bajo] · **Ubicación** `scripts/smoke_test.py`, `engine/database.py::set_flow_config`

Llama a `set_flow_config('03_folder_inventory', ...)`, que reescribe `configs/03_folder_inventory.json`, un archivo versionado. **Reproducido:** tras ejecutar el smoke test, `git status --porcelain` marcó ese archivo como modificado. Se restaura con `git checkout --`.

Contribuye `engine/database.py::set_flow_config`, que usa `Path.write_text` sin `newline='\n'`, produciendo CRLF en Windows.

Recomendación. Que el smoke test use un flow de prueba desechable o un `AUTOMA_DATA_ROOT` temporal. Añadir `newline='\n'` a las escrituras de `set_flow_config` y `set_secret`. **Prioridad: 26.**

R-27 · Dos verificadores de pre-commit no tienen equivalente en CI

Severidad [bajo] · **Ubicación** `.pre-commit-config.yaml`, `.github/workflows/`

`ruff-format` y `markdownlint-cli2` están declarados como hooks locales pero **no se ejecutan en ningún workflow**. Verificado: `grep -rn "ruff format" .github/workflows/` no devuelve nada. Un colaborador sin `pre-commit install` puede subir código con formato distinto sin que nada falle.

Además, `markdown-docs.yml` solo se dispara en `pull_request` con `paths: '**/*.md'`: un push directo a `main` con un enlace roto no lo detecta.

Recomendación. Añadir `ruff format --check` a `ci.yml` y disparar `markdown-docs.yml` también en `push` a `main`. **Prioridad: 27.**

5. Decisiones que requieren validación humana

No son defectos: son puntos donde el código toma una decisión que alguien debería confirmar conscientemente.

#	Decisión	Dónde	Por qué requiere confirmación
D-1	Mantener <code>plugins/analyzers/vision_model_analyzer.py</code> en el repositorio	<code>plugins/</code>	222 líneas capaces de enviar capturas de pantalla a un endpoint externo, sin ningún flow que las use. Contradice aparentemente el posicionamiento «sin IA», aunque en la práctica es inalcanzable
D-2	<code>browser.fill_form</code> con <code>headless=False</code> por defecto	<code>actions/browser_form.py</code>	Abre una ventana visible que interfiere con el trabajo del operador. Es intencional (la demo es el producto) pero sorprende en uso programado
D-3	<code>_pick_record</code> marca el registro como usado antes de llenar	<code>actions/browser_form.py</code>	Prefiere perder un registro a repetirlo. No está documentado en el README del flow
D-4	El webhook es síncrono	<code>app/server.py::do_POST</code>	Una petición contra un flow largo bloquea la conexión. <code>/api/run</code> sí es asíncrono
D-5	Los eventos se escriben dos veces (JSONL + tabla <code>events</code>)	<code>engine/logger.py</code>	Duplica el volumen de la traza. Es redundancia deliberada por resiliencia
D-6	<code>_persist()</code> tras cada paso	<code>engine/orchestrator.py</code>	Una escritura de disco y una transacción por paso. Correcto para flows cortos, caro para flows largos
D-7	El JSON Schema no se aplica en runtime	<code>engine/manifest_schema.py</code>	Solo lo usa la CI. Un manifest inválido llega hasta el motor y falla con <code>KeyError</code>
D-8	<code>release.yml</code> no ejecuta <code>pytest</code>	<code>.github/workflows/release.yml</code>	Corre <code>validate_project.py</code> pero no la suite. Un tag sobre un commit rojo produciría instalador
D-9	Los datos sobreviven a la desinstalación	<code>engine/paths.py::data_dir</code>	<code>%LOCALAPPDATA%\Automa</code> no lo borra el desinstalador. Incluye todas las capturas de pantalla

6. Lo que está bien y hay que proteger

Estas propiedades son las que hacen el sistema sólido. Alguien las romperá si nadie dice que importan.

Propiedad	Dónde	Por qué protegerla
Separación puro/impuro	<code>actions/browser_extract.py</code>	Permite 91 % de cobertura sin navegador. Debería ser obligatorio en toda acción nueva
<code>dry_run</code> en las acciones de UI	<code>actions/ui.py</code>	Hace testeable lo que por naturaleza no lo es, y permite probar sin efectos

Cotas explícitas y declaradas	Casi todas las acciones	<code>truncated, links_truncated, stdout_truncated...</code> Nunca truncado silencioso
Degradación con motivo legible	OCR, portapapeles, Playwright	<code>status: "unavailable"</code> con instrucciones por sistema operativo, en vez de un crash opaco
Determinismo declarado y sostenido	<code>crawl_pages, check_urls, resolve_links</code>	Orden de aparición, sin aleatoriedad. La única fuente de no-determinismo es <code>random.choice</code> en el flow 07, y es intencional
<code>policy.summary()</code> guardado en el histórico	<code>engine/orchestrator.py</code>	Se puede auditar, corrida a corrida, cuáles corrieron sin restricción
Comentarios que explican el porqué	<code>paths.py, introspection.py, template.py, ui.py, server.py</code>	Documentan decisiones y modelos de amenaza, no líneas de código
CWE citados en el código	<code>app/server.py, actions/ui.py, actions/system.py</code>	Cada control dice qué cierra. Práctica poco habitual y muy útil para auditar
12 capas de hardening del CI	<code>.github/workflows/</code>	SHA pin verificado por parser YAML propio, <code>detect-secrets</code> sobre 50 commits de historial, checksum del propio <code>actionlint</code>
Separación raíz lectura / raíz escritura	<code>engine/paths.py</code>	Resolvió un fallo real de despliegue en la arquitectura, no con un parche
SQL 100 % parametrizado	<code>engine/database.py</code>	Sin una sola concatenación
<code>_safe_folder</code> en las 7 rutas	<code>app/server.py</code>	Una única función, aplicada consistentemente
Los 4 controles de <code>/file</code>	<code>app/server.py</code>	Incluido el <code>+ os.sep</code> que cierra el bypass por prefijo hermano
El validador comprueba coherencia de política	<code>scripts/validate_project.py</code>	No solo sintaxis: verifica que ningún paso use una acción que su propio manifest prohíbe
Los flows web apuntan a HTML locales por defecto	<code>27 context.example.json</code>	La demo es determinista y sin internet. Cero tráfico de red por defecto
Cotas de dependencias con motivo escrito	<code>pyproject.toml</code>	«Primera versión sin CVE conocida al corte de la auditoría 2026-06-01»
Fixtures que aíslan sin parchear	<code>tests/conftest.py</code>	Cambian el cwd; las pruebas ejercitan el mismo camino que producción
El sistema crece por casos, no por refactor	<code>flows/</code>	27 casos construidos con las 36 acciones existentes. El motor no se toca

7. Plan sugerido, por esfuerzo e impacto

Ninguna acción se ejecutó.

Rápidas y de alto impacto (< 1 hora cada una)

#	Acción	Hallazgo
1	Añadir <code>actions.browser_extract</code> a <code>hiddenimports</code>	R-01

2	Cambiar <code>if raw.get(...)</code> por <code>is not None</code> en <code>load_manifest</code>	R-05
3	Exigir token en los GET cuando <code>AUTOMA_PANEL_TOKEN</code> esté definido	R-02
4	Reconfigurar <code>sys.stdout</code> a UTF-8 en <code>runner.main()</code>	R-06
5	Regla en <code>validate_project.py</code> : exigir <code>allowed_actions</code>	R-03
6	Importar <code>root_dir</code> de <code>engine.paths</code> en <code>catalog.py</code>	R-11
7	Sincronizar <code>requirements.txt</code> y los entry points	R-19, R-20
8	Añadir <code>ruff format --check</code> a <code>ci.yml</code>	R-27

Medias (medio día cada una)

#	Acción	Hallazgo
9	Mover el arranque del scheduler a <code>run_server()</code>	R-16, R-17
10	Aplicar <code>run_locks</code> en las rutas del panel y del webhook	R-07
11	Índices sobre <code>runs.flow_id</code> , <code>steps.run_id</code> , <code>events.run_id</code>	R-08
12	Unificar los dos motores de condiciones	R-14
13	Declarar las divergencias del cron en la ayuda del panel	R-22
14	Pruebas de <code>_pick_record</code> , <code>find_text_in_image</code> y del CLI	R-06, R-23

Mayores (varios días)

#	Acción	Hallazgo
15	Timeout real por acción	R-09
16	Rutina de retención configurable	R-08
17	Extraer CSS y plantillas de <code>app/server.py</code>	R-25
18	Generar y versionar el lockfile, e integrarlo en la CI	R-18

Documentos relacionados: [11 · Seguridad](#) · [12 · Pruebas y calidad](#) · [13 · Despliegue y operación](#) · [14 · Solución de problemas](#) · [17 · Resumen ejecutivo](#)